

CropMind

AI-Powered Smart Farming Assistant

Intern ID: TBI-26100746

Name: Deepanshi Agarwal

University Name: Graphic Era Deemed To be University

Evaluation: TBI-GEU Internship Project

The Problems We Are Solving

- **Scattered Data and Info**

Farmers have to check multiple websites and tools to find soil moisture readings, weather forecasts, and APMC market prices. It is slow and confusing.

- **Slow Help for Sick Crops**

Getting advice on plant diseases or soil issues usually takes days, which can lead to crop damage and financial losses if not handled quickly.

- **Difficult Interfaces**

Many farming apps have complicated layouts that are hard to navigate. There is a lack of simple, conversational tools to answer day-to-day questions.

- **Our Solution (CropMind)**

We put live soil data, market price trackers, and an AI diagnostic assistant on one simple, user-friendly dashboard.

Our Tech Stack and Why We Chose It

Frontend Client

- **React 19 Framework**

Helps build responsive pages and makes state management easy across different components.

- **Vite 7 Build Tool**

Gives fast reload times during development and makes the overall project build very quickly.

- **Vanilla CSS Variables**

Allows complete control over the layout, colors, animations, and dark/light modes without relying on heavy UI frameworks.

Backend, Database & AI

- **Node.js & Express 5**

Handles backend API requests quickly and securely communicates with the database and AI models.

- **MongoDB & Mongoose**

Perfect for storing unstructured telemetry logs and crop entries. Very easy to update fields as requirements change.

- **Google Gemini 2.5 Flash**

Generates helpful farming advice and diagnostic checklists quickly (usually under 2 seconds).

Web Application Features

- **Farming Telemetry Dashboard**

Displays live metrics: Soil moisture (72%), rain probability (40%), and wholesale APMC market price trends.

- **AI Agronomist Diagnosis**

Allows users to submit crop symptoms, soil type, and watering details to receive an organic treatment checklist.

- **Floating Assistant Bubble**

A chat window available on all pages for asking everyday farming questions, backed by the AI model.

- **Theme Toggles**

Supports dark and light mode preferences, with custom CSS styling and responsive transitions.

Live Website Link:

<https://crop-mind-chatbot-frontend.vercel.app>

Top 2 Backend APIs

1. AI Symptoms Diagnostics

POST /api/ai/diagnose

- What it does: Takes the crop name, symptoms, soil type, and watering frequency.
- How it works: Sends these details to Gemini, telling it to return specific JSON keys representing diagnosis, severity, treatment, and prevention.
- Security: Checks that the request has a valid token and restricts users to 10 requests per 15 minutes to prevent system abuse.

2. AI Chatbot Proxy

POST /api/chat

- What it does: Connects the chatbot on the website to the Gemini AI model.
- How it works: Keeps the secret API keys safe on the server (the browser never sees them).
- Formatting: Instructs the AI to focus strictly on Indian farming and keep replies brief (under 8 bullets).
- Fallback: If no Gemini key is set, the API returns a mock message instead of crashing.

Database Selected & Schemas

Selected DB: MongoDB

▪ Flexible Schemas

We can store sensor logs and crop datasets without setting up rigid, complex tables.

▪ Mongoose Integration

Provides simple data validation rules (like marking fields required and setting up enums) directly in the code.

▪ Offline Developer Fallback

Includes a mock in-memory handler to let developers run the project locally if MongoDB is disconnected.

Mongoose Document Models

1. Crop Schema:

- `name: String (required)`
- `season: String (enum: ['Kharif', 'Rabi', 'Zaid', 'Annual'])`
- `water: String (enum: ['Low', 'Medium', 'High'])`

2. Telemetry Log Schema:

- `time: String (required)`
- `type: String (enum: ['system', 'info', 'success', 'warning'])`
- `text: String (required)`
- `timestamps: true`

AI Feature & Gemini LLM Integration

- **Used LLM: Google Gemini 2.5 Flash**

Chosen for its fast processing speed (usually under 2 seconds) and detailed responses.

- **Use Case 1: Smart Crop Diagnostics**

Evaluates crop type and symptoms to suggest treatments and preventions, formatted as clear checklists.

- **Use Case 2: Natural-Language Assistant**

Answers conversational questions regarding seed choices, NPK fertilizer balances, and optimal sowing times.

- **Strict JSON Output**

We use strict system prompts to force the model to reply only in parseable JSON. This prevents page layout breaks and raw text display issues.

Hosting and Cloud Services

- **Frontend: Hosted on Vercel**

Hosts the React client. Rebuilds the frontend page automatically whenever changes are pushed to GitHub.

- **Backend: Hosted on Render**

Hosts our Node/Express server. Safely injects keys (like GEMINI_API_KEY) so they are never exposed to the public frontend.

- **Database: Managed on MongoDB Atlas**

Deploys our database cluster to the cloud, managing connections and automatic document indexing.

- **Local Fallback Support**

If the cloud servers are offline or keys are missing, the project has built-in local fallback modes for smooth testing.

Project Links and URLs

Live Website (Frontend)

<https://crop-mind-chatbot-frontend.vercel.app>

Live Backend API

<https://cropmind-chatbot-backend.onrender.com>

Developer API Portal (Docs)

<https://cropmind-chatbot-backend.onrender.com/docs>

GitHub Repository

<https://github.com/deepanshi-code/CropMind-Chatbot>

Internship Reflection

- **Full-Stack Architecture**

Learned how to hook up a React client to a Node/Express backend, and sync visual components with MongoDB collections.

- **Security Best Practices**

Understood why sensitive keys (like API secrets) must remain on the server, and how to verify request access tokens.

- **Working with AI Models**

Gained experience directing LLM outputs via strict formatting rules (JSON formatting) to avoid parsing glitches on the frontend.

- **Cloud Deployments**

Successfully deployed frontend, backend, and database servers across Vercel, Render, and Atlas platforms.

- **My Experience**

A great learning experience creating a practical tool that addresses real-world farming challenges.